

Reconciling the irreconcilable? A software development approach that combines Agile with Formal.

José Gonalo A. Oliveira Basto da Silva
Critical Software, S.A.
Parque Industrial de Taveiro, Lote 48
3045-504 Coimbra, Portugal
jsilva@criticalsoftware.com

Paulo Rupino da Cunha
Departamento de Engenharia Informtica
Universidade de Coimbra
3030-290 Coimbra, Portugal
rupino@dei.uc.pt

Abstract

Unstable software requirements are common and call for Agile practices. On the other hand, quality standards and recommendations, to which companies must adhere to play in selected markets, enforce formality in the development process. We present an approach, tailored for a real company, which reconciles these extremes. We used as inputs the original software development process and its formality constraints, the difficulties felt by the company, and various agile approaches. The outcome has been tested in various cases with encouraging results regarding awareness over project progress, requirements management, control of allocated effort and time, and product quality. Although the approach was customized for a specific company, it may be insightful to others facing similar challenges.

1. Introduction

Critical Software develops “innovative and reliable solutions, systems and information technologies for enterprise Customers and partners’ business critical systems”. It was started by three Ph. D. students from the University of Coimbra in 1998, but it quickly expanded to more than one hundred staff with offices in Portugal, USA and UK [11][13]. Its Customers include NASA, the European Space Agency, Alcatel, Vodafone, Siemens, among other distinguished companies and public sector institutions all over the world.

The strong will to make a difference in the international market, where other competitors held a solid position, led Critical’s top management to a strong commitment to Quality. This strategic decision and its implications earned the company several prizes and distinctions [11] and worldwide recognition, patent in the numerous case studies about it.

Critical Software’s Quality Management System (QMS) conforms to several standards, namely ISO 9001’s business independent guidelines, norms that define software life cycle processes (ISO 12207), Software Process Improvement and Capability mEasurement (SPICE), European Community for Space Standardization (ECSS) and, finally, National Aeronautics Space Agency – Software Engineering Laboratory (NASA-SEL) recommendations.

According to ISO 12207’s recommendations, there is a Projects Life Cycle Management (PLCM) process, which defines the types of projects that are developed at Critical Software and their specific life cycle models. Currently the following types of projects are considered: software development; product/process evaluation; product selection and deployment; optimization; product development and support; maintenance; and generic consulting projects [10]. For software development projects, the PLCM hands over to the Software Development Process (SDP). The SDP defines one generic life cycle, which is comprised of six phases: system requirements engineering phase; software requirements engineering phase; design phase; validation phase; acceptance phase; and support and maintenance phase. Additionally, four tailorings exist for four different contexts: prototyping; new development; high-reuse; and enterprise application integration (EAI).

In the first few years, most of Critical’s clients came from the Space sector. Their requirements were fairly stable and development followed the plans without major changes. However, as Critical got more involved with the industrial sector, specific difficulties emerged, combined with new problems related to the growth of the company. These can be summarized as follows:

- **Evolving requirements.** Industrial Customers are constantly changing their requirements in order to keep up with evolving business needs [1][5][17].

This turbulence is, nowadays, a fact that must be assumed and dealt with.

Also, Customers often have only a rough idea of what they want, or they are not specific enough when stating their needs. According to Critical's experience, sometimes Customers even make conflicting declarations when asked about the same requirement at different times.

- **Lack of Customer involvement.** The Customer pays for the project, therefore, as far as they are concerned, it is the project team's job to develop the solution. The Customer rarely allocates any effort for support activities. However, it is known that, the less support is given by the Customer, the higher are the chances of project failure [8][17][19][20].
- **Tight deadlines, high demands, low tolerance and low budgets.** Due to the worldwide economic slowdown, Customers are less tolerant to failure. Not only the pressure and demands rose over the last few years, but also the budgets have decreased

due to the aggressive competitiveness from other software houses.

- **Miscommunication between developers and Customers.** When talking to each other, Customers and developers often use their own jargon, which is frequently not common-ground for both parties [14]. On the other hand, excessively informal discussions of requirements between developers occasionally lead to different and imprecise understandings of the actual objective [16].

This is the context in which a highly formal software development process, strongly influenced by several quality standards, must be used to meet requirements from Customers. Additionally, top management demands agility and calls for metrics to help monitor project progress continuously and in a more effective way (Figure 1).

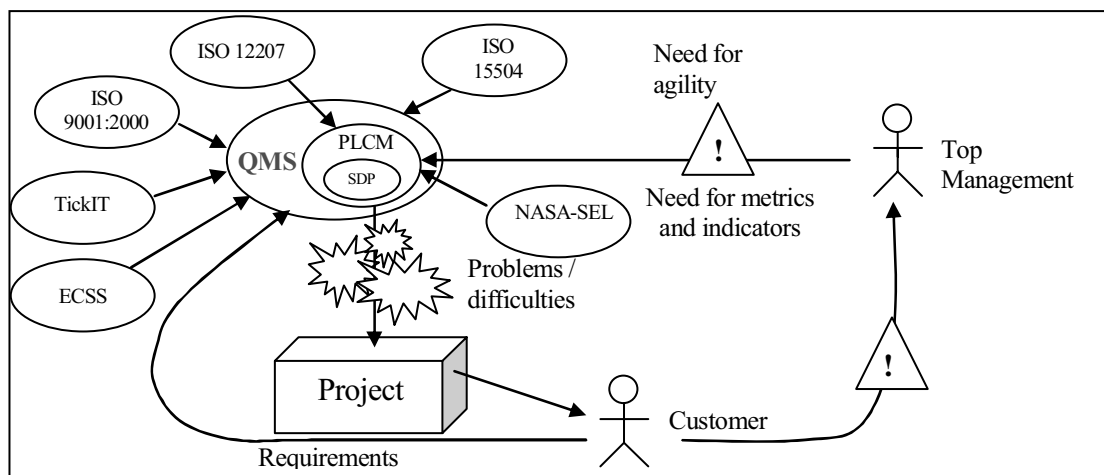


Figure 1 - Context of application of the software development process

Due to its nature, Critical Software's formal software development process can't easily be adjusted ad hoc. Apparently minor changes may easily trigger unexpected overhead and non conformities, eventually leading to falling Quality Audits. On the other hand, due to the aggressiveness of deadlines and the high probability of missing them, project managers tend to reject using agile practices, in fear of not being in complete control of the situation. The required metrics, on the other hand, must be collected without increasing project overhead significantly.

Although most industrial projects are claiming for the introduction of agile practices, none of the existing methodologies can be used, unmodified, in Critical's context. On one hand, there are the various

quality certifications, which must be maintained to be able to play in selected markets. On the other hand, the company wishes to keep several procedures, templates and tools that have already proven their effectiveness and are embedded into the company's workflows.

The solution points to an apparent paradox: trying to reconcile the agility of recent methodologies with the necessary degree of formality imposed by quality management standards and recommendations, to come up with a process capable of managing evolving requirements to ensure the delivery of software to the Customer on time, on effort and on quality. To this end, agile methodologies were studied and selected

practices were introduced in a new tailoring of Critical's software development process.

A new life cycle model was created to allow iterative and incremental development without the need of formal milestone meetings in between. The Enterprise Architect CASE tool [22] was tailored to help manage projects requirements more effectively through the use of relationships matrices and automatic document generation. A new testing tool is being evaluated, so that unit tests will be widely used, with as little overhead as possible.

To ensure rigor and validity in the definition of Critical's custom agile approach to software development, action-research is being used, since it is recognized as one of the few research approaches that can be legitimately employed to study the effects of specific alterations in systems development methodologies [2][3][12]. Action-research possesses a cyclic nature: an intervention is planned, the corresponding action is taken, and a critical reflection on the results is carried out, providing insight for adjustments that can be made before the next cycle. The researcher has been involved in the process as the Quality Assurance Engineer for four projects. Three of the projects comprised one Project Manager and two Project Engineers and the other project comprised one Project Manager and three Project Engineers. So far, the custom agile methodology that Critical is developing has been used in three cycles.

As illustrated in Figure 2, the first cycle encompassed one project. Its findings had significant impact and required reflection and planning for a second cycle. The second cycle spanned two projects and more interesting findings emerged, resulting in the need for more reflection and planning for a third cycle. Meanwhile, one of the projects ended, while the other three are still on-going. The present article describes the process in its third cycle and all related findings.

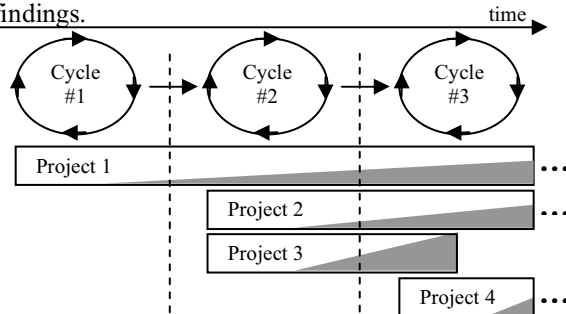


Figure 2 – Action-research cycles and related projects in the study, so far.

The remainder of this paper is organized as follows: section 2 describes the proposed software development process. Section 3 discusses some preliminary results, by comparing two similar projects, one of which used the old formal process and the other the new, more agile, approach. Finally, Section 4 presents some conclusions. Due to space constraints, the state-of-the-art on agile methodologies is not included in this paper.

2. An approach that balances agile with formal

Since it would not be feasible to examine all agile methodologies and their practices in the required time frame, we sought inspiration in three of the most popular and documented methodologies: eXtreme Programming, Adaptive Software Development and Scrum. In a second round of analysis, some practices used in these approaches were discarded, mainly due to the very small size of the teams: pair programming and daily stand-up meetings were ineffective in groups of two programmers. Another interesting fact is that the scrum sprint backlog was not efficiently implemented until the third project, and it only returned effective results in three of four projects because of one team's lack of discipline.

In order to comply with the overarching norms defined in the software development process life cycle model for which Critical Software holds a certification, the agile approach was designed as a particular tailoring for use in cases of unstable requirements. It is composed of four phases, as illustrated in figure 3:

- **Requirements Phase.** Dedicated to project preparation and specification of high-level requirements.
- **Design, Engineering and Validation Phase.** Where development occurs, iteratively.
- **Acceptance Phase.** Where acceptance tests are performed by the Customer before closure.
- **Operations and Maintenance Phase.** Eventual bugs and fixes are addressed in this period.

Phases are delimited by milestones where key objectives and tasks are validated. It is during the milestone meetings that project members evaluate whether or not the project team is prepared to move on to the next phase.

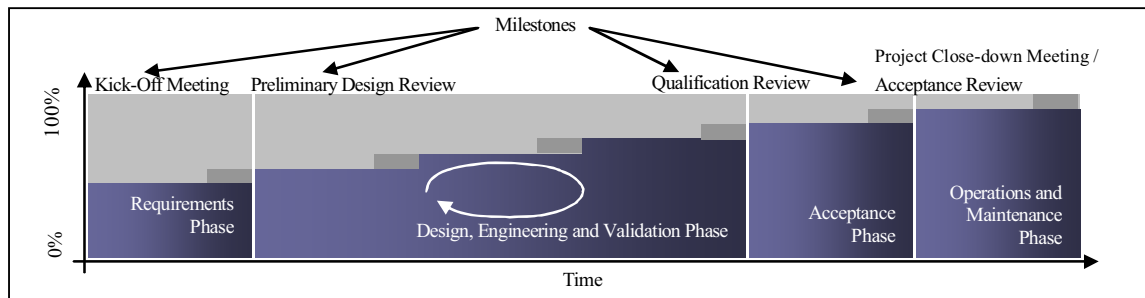


Figure 3 - Agile Software Development Process life cycle model.

The project initiates with a Kick-Off Meeting, where the Project Manager formally presents major artifacts (e.g. project proposal details, project master plan, project folder, main roles and responsibilities, and quality assurance plan). All milestone meetings are monitored by a Quality Assurance Engineer, whose role is to ensure that quality tasks are performed and objectives are met.

It is important that the whole project team is identified. It consists not only of Critical's staff, but also of Customers' stakeholders, co-responsible for project success. Customers' liaisons should be able to help in requirements specification, decision making and final approval. They are in charge of making sure that Customers' tasks with impact on the project progress are performed within a reasonable period (e.g. three days in a one month project). Penalty clauses are included in the project proposal to account for delays.

The Requirements Phase main objective is to identify functionalities desired by the Customer. Customers tend to understand functionalities better than lower level software requirements [7][17]. Functionalities should be described by simple and unambiguous phrases using the SMART technique [18]:

- **Specific.** Functionalities should be unambiguous and clear. The writer should specify things in a clear, consistent, simple way, and at an appropriate level of detail.
- **Measurable.** Once the functionality has been implemented, it should be verifiable, in order to ensure that the requirement was met.

- **Attainable.** By an attainable requirement we mean it is possible physically for the system to exhibit that requirement under the given conditions. Some requirements may be beyond the bounds of human knowledge. Others may have theoretical solutions but be beyond what is currently achievable."
- **Realizable.** Functionalities should be feasible under the described conditions, given resources and time frame.
- **Traceable.** Requirements should be traceable from the initial discussion with the Customer, subsequent formalization into a document, through design, implementation and test, to ensure coding and testing are exhaustive.

SMART requirements help avoiding diverging understandings from Customers and project developers regarding the same issues, while promoting the ability to grasp new and abstract concepts quickly. Whenever needed, the "I Know It When I See It" (IKIWISI) technique should also be applied, resorting to working prototypes to help Customers shed light over ambiguous requirements[5][21].

Requirements are managed in a CASE tool called Enterprise Architect. This tool helps the project team keep relationship matrices between functionalities and lower level requirements (Figure 4), for analysis of impact of change requests ("what is the effort involved in the requested change?"), and between requirements and tests for analysis of requirements coverage ("are the tests covering all requirements?") [14][15].

Relationship Matrix

Source: Use Cases Catalogue

Type: UseCase

Link Type: Realisation

Profile: UC - UR assoc

Target: Requirements

Type: Requirement

Direction: Bi-Directional

Refresh

	Funcional: Alteração	Funcional: Alteração	Funcional: Criação de	Funcional: Detalhe de	Funcional: Listagens:	Funcional: Listagens:	Funcional: Registo de	Funcional: Registo de	Integração: Criação de	Integração: Registo de	Integração: Sistema de	Integração: Sistema de	Integração: Sistemas	Manutenção: Manual	Manutenção: Monitor	Manutenção: Optimiz	Manutenção: WOW F	Refactoring: Cache	Refactoring: Detalhe	Refactoring: Flag can	Refactoring: Grupos	Refactoring: Registo	Refactoring: Registo	Refactoring: Remove
Funcional::Listagens: Categ...					X			X		X														
Funcional::Listagens: Geraçã...						X									X		X							
Funcional::Registo de Acçõe...							X				X	X		X	X									
Funcional::Registo de Acçõe...								X						X		X		X						
Integração por Email: LIC-42...								X	X					X	X	X	X	X				X		X

Figure 4 - Enterprise Architect relationship matrices between functionalities and lower level requirements.

The Enterprise Architect case tool is capable of handling several attributes such as requirements priority, risk, difficulty, effort, type (functional, non-functional, interface, performance, printing, reporting, testing, validation), status (proposed, draft, approved, implemented, and reviewed) and author (member of the Customer team that requested the functionality), it keeps an history of changes (through comparison of requirements against the baseline) and it is also capable of automated document generation. In this case, the requirements specification document, in Word, Rich Text or HTML format.

When all functionalities have been documented, this information is reviewed with the Customer using a Powerpoint presentation. A formal specification document should be signed after clarifying eventual outstanding issues.

Before the beginning of the next phase, a Preliminary Design Review (PDR) meeting held by the project team. This meeting is conducted by the Project Manager and monitored by a Software Quality Assurance Engineer. The PDR is a milestone to check whether or not goals defined at the beginning were met: were the expected deliverables produced and reviewed as planned? Was reuse analysis performed? Are there any process improvement suggestions? Were test cases defined? Were the matrices produced? And, in conclusion, is the project prepared to move on to the next phase?

Figure 5 illustrates the procedures just described, from project beginning until the Preliminary Design Review milestone meeting.

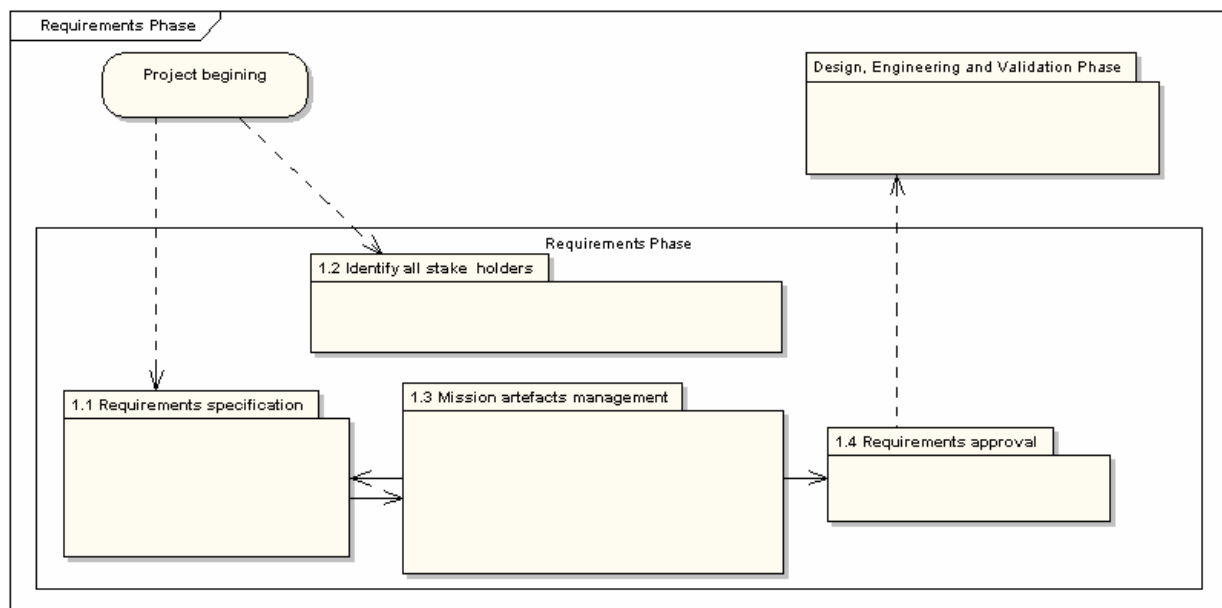


Figure 5- Requirements phase

Development takes place, iteratively and incrementally, in the Design, Engineering and Validation Phase (Figure 6). An increment is composed of one or more iterations and results in a working fragment of the desired final product. The Design, Engineering and Validation phase is composed of three major sub-phases:

- **Adaptive Cycle Planning.** This sub-phase marks the beginning of a new iteration. The first activity is to schedule a workshop with the Customer, who is asked to define a timeframe within which a working increment must be available. This is called timeboxing [6][17]. Timeboxes (iterations) should have duration of no less than one week and no longer than four weeks. Short iterations lead

to a higher focus on the requirements by the development teams [8][17]. They are forced to develop less and quicker. As a direct result, the outcome is easier to maintain, test, and eventually change [6]. According to [19], shorter time frames increase project success. This practice not only assures higher focus on objectives, but also quick steering in case of deviations. All the functionalities identified in the previous phase are laid down on the table and the Project Manager, Senior Engineer(s) and the Customer Representative(s) prioritize them according to their risks and dependencies.

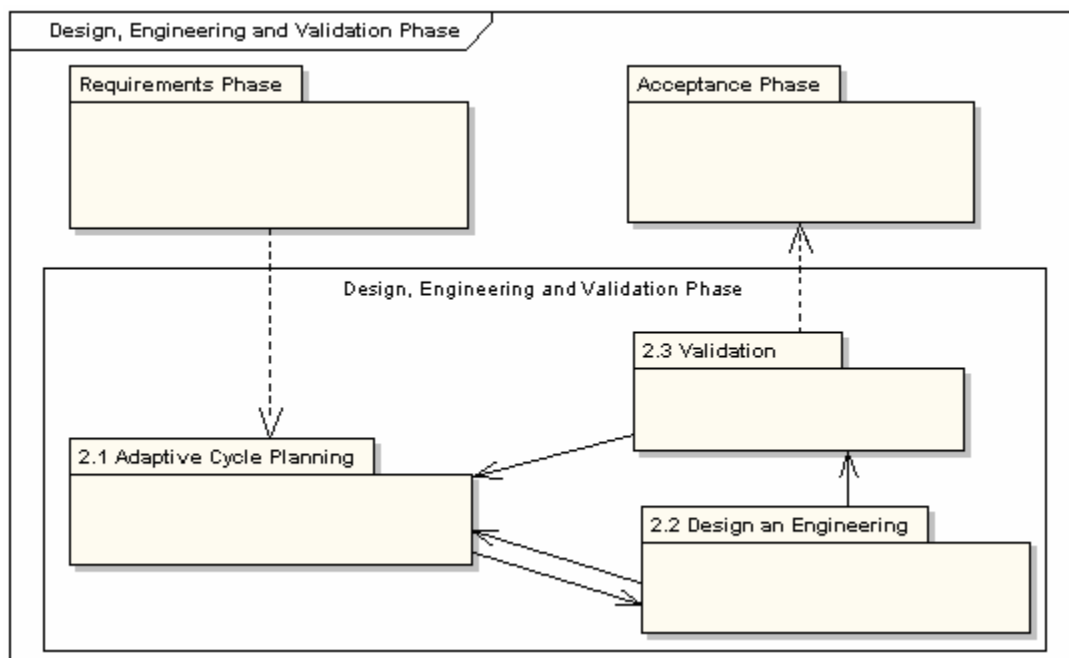


Figure 6 - Design, Engineering and Validation phase

At this stage, the Project Manager and the Senior Engineers perform a brainstorming in order to design detailed components. This is a critical step, because good component engineering allows the implementation of simple, yet robust solutions, with high and yet effective rates of reuse, which allows easier modification of the code in case of change requests [17]. Functionalities and components are related in a matrix using the Enterprise Architect tool. Based on the brainstorming results, the project manager creates a detailed iteration plan to guide the development team in the implementation of all the components underlying the chosen functionalities for the current iteration.

In the beginning of the each iteration, the Software Quality Assurance Engineer creates a backlog for the current iteration requirements – a list of all outstanding work. This should then be updated on a daily basis, keeping track of the effort spent. The use of a backlog is a practice taken from SCRUM [17].

- **Design and Engineering.** Actual development, according the detailed project plan, starts in this sub-phase. Remaining technical details are clarified on an on-going basis. As implementation advances, test cases are defined and executed in parallel using an automated testing tool as well as manual tests whenever automation is not possible or reasonable.

Once an iteration has started, requirements under development cannot be changed. It is the Project Manager's role to manage communication with the Customer and restrain change requests. If the Customer demands modifications to the original requirements, its impact is analyzed by the Project Manager using the functionalities vs. components matrix. Since functionalities are characterized in terms of effort, eventual financial and time implications can be accurately reported back for approval. Nevertheless, the current iteration is not affected, since changes will only be accounted for in following iterations or increments. The Project Manager has also the concurrent responsibility of preparing the next iteration, following the activities described for the Adaptive Cycle Planning, in order to ensure that the programmers are not idle in-between sprints.

The implementation activity is supported by Cruise Control – an open source, highly configurable continuous integration tool – that provides automated build construction, e-mail notification on error detection, automated execution of test scripts and metrics such as percentage of successful builds.

- **Validation.** Upon conclusion, the increment is subjected to system tests and presented to the Customer at a workshop, where, together with the project manager, an evaluation is done to check if it meets the expected results. Based on the conclusions, both parties decide whether the increment is ready to be deployed at Customer's premises for acceptance tests.

If the increment needs no further iterations, the Project Manager fills a Reuse Sum-up document describing which existing components were reused, and which new ones can be added to the company's Repository. Using this information, the Reuse Administrator can report to top management on effort saved and invested across all projects.

The next step in the Qualification Review Milestone is to hold a meeting to confirm that all increments are implemented and deployed at Customer's premises, that a snapshot of the physical architecture of any databases related to the project is recorded, and that acceptance tests are appropriately specified and ready to be used in forthcoming Acceptance Phase.

As shown in Figure 7, each software increment is tested and evaluated in a formal meeting with the Customer. If any problems are detected they must be fixed and the tests re-applied.

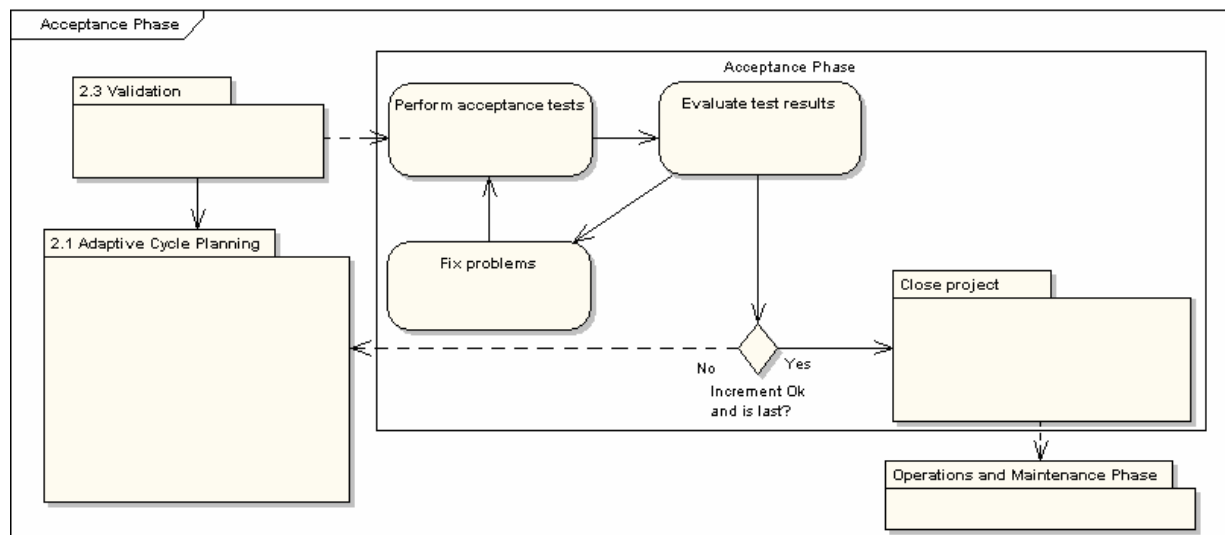


Figure 7 - Acceptance phase

If the deployed increment was not the last one, the project continues in the Adaptive Cycle Planning task of the Design, Engineering and Validation phase. Upon completion of all increments, the Customer acknowledges project conclusion by signing an

acceptance form. That same day, the team holds a Project Close Down meeting, where the whole project is reviewed and process improvement suggestions are discussed. The Project Manager records all related information into an optical disc for safe keeping and

configuration management purposes. A liaison is assigned to handle any future Customer requests during the Operations and Maintenance Phase.

3. Preliminary results

We have been experimenting with the described Agile Software Development Process in four projects. Initially, only a subset of the practices and tools were used, to draw partial conclusions and refine different ideas. Recently, the full proposal has been used to manage a delicate project – still under way – to which we will refer as X2.

Project X2 is the evolution of a previously released project X1 which was managed with the old formal

methodology and suffered a series of problems (see Table 1). It consisted of a workflow system designed to manage trouble tickets and work orders throughout the whole lifecycle - from initial issuing to task conclusion - while ensuring user and time tracking as well as recording of results.

For project X2 the requirements called for a more ambitious version of this workflow system, capable of being shared by several other companies across Europe, which are part of the same financial group of the initial company.

Table 1 shows how similar issues were played out in project X1 and in project X2.

Table 1 – Comparison between Project X1 and Project X2.

	Project X1	Project X2
Requirements	Neither the development team nor the Customers could clearly understand the requirements specification. They were poorly defined. Both parties though there was tacit agreement on what the requirements were.	Requirements were defined with Customer's involvement. The team used the practices described in the new agile process to specify requirements.
	The Requirements Phase finished on time, as specified in the project plan, albeit requirements later revealed unclear and imprecise.	The Requirements Phase failed to meet the Preliminary Design Review (PDR) milestone according to the plan, but the understanding of the requirements proved more effective.
Quality Assurance	No traceability matrices between functionalities and lower level requirements were used. The project manager was unable to correctly evaluate the impact of change requests in terms of effort to address them.	The use of the Enterprise Architect CASE tool helped managing supporting information, relationship matrices and automatic document generation.
	The project was reviewed and lessons learned only at the end, in a single Project Close Down meeting.	The project is reviewed at the end of each iteration. Problems are detected earlier and appropriate corrective measures implemented.
Testing	Tests were defined on the fly, during implementation, and executed manually. As soon as the project started to miss deadlines, project members started to skip testing activities in order to speed up implementation. The project had an unacceptable number of bugs.	Acceptance tests were defined before the Preliminary Design Review milestone. The project used unit tests from the previous version and new ones were developed for the current version. Tests were performed during implementation, and integration tests executed after the conclusion of each iteration.
	The testing environment was not a concern. Neither the environment nor the data were realistic. The executed tests passed in spite of the project having an unacceptable number of bugs.	The project team prepared a realistic testing environment and set up a database populated with real data. Tests were planned and executed rigorously.
Team Empowerment	The project members were not empowered. They had strict orders from the project manager. New ideas were not welcomed.	The team contributed positively with ideas. Morale and enthusiasm towards the project are higher than in the previous version.
Change request	Customer change requests were managed informally, using e-mails and phone calls. No change log was maintained. In some cases, the Customer even requested the team to change things back to the original state.	A change request procedure was defined in the contract. Time and effort estimates are calculated for each one and formal approval is called for.
Monitoring	The project progress was not properly monitored. Top management only knew about project delay near the end.	The Software Quality Assurance Engineer keeps a backlog of each iterations. Top management has access to progress data independent from the Project Managers point of view. This backlog also helps predict how much longer the project is going to take before all increments are finished, based on the project teams performance.

A striking question regarding the above comparison is: how could Project X1 suffer so many problems in a software development company with a quality-certified formal process? The answer lies in the overhead of the process itself, which renders it unusable in cases of unstable requirements and highly aggressive deadlines. Being unable to apply the process, the Project Manager is left without reliable instruments and ends up improvising as issues arise.

Planning small increments in project X2 allows the development team to be more focused on immediate implementation needs and the Customer is able to test the project earlier. Conversely, in project X1, the bugs were found only when the full project was deployed, after all functionalities were implemented. Using automatic unit tests and early integrations of the project at the Customer's facility, the project team is able to address problems at an early stage. As project X2 progresses, a reduction in change requests and risk is being noticed as is an increase in Customer satisfaction.

The monitoring of project progress also improved in terms of efficiency and effectiveness. The Software Quality Assurance Engineer ensures an autonomous and independent project monitoring in parallel with the Project Manager.

According to the latest report to top management, project X2 is presently fully on quality, which means that all expected quality records are being produced as required by Critical Software's Quality Management System. It is however neither on time nor on effort due to late approvals of change requests by the Customer, who acknowledges this responsibility.

The current project is being released seamlessly. So far, two of the three increments were deployed in the production line with zero bugs. In spite of the small delays which resulted in response to Customer's changing needs, their expectations are being fulfilled, and the team morale is high.

Project X2 is also becoming notorious inside Critical Software, raising curiosity about the Agile Approach among other Project Managers.

4. Conclusion

We presented a software development process that is, simultaneously, agile enough to deal with unstable requirements and formal enough to meet quality certifications. It has been custom tailored for a company whose traditional clients demand strict methodologies to handle their stable but critical requirements; but whose newer clients demand flexibility to change requirements without missing aggressive deadlines. The approach has been tested in

several real cases with encouraging results. In those cases where the deadlines were missed, the Customer acknowledges the responsibility for not having responded promptly to the development team.

The proposed approach meets most of the challenges raised in the introduction, namely the need for an adjusted iterative and incremental approach which allowed facing evolving requirements in a responsive way. The Customer's participation in requirements specification, reviews and workshops, not only in key moments of the project but also between sprints gave the Customer a sense of ownership over what was actually being produced. The fact that the project was being delivered incrementally allowed the Customer to see results sooner, raising its enthusiasm. It also allowed the Customer to sense Critical's speed of development, allowing request for changes and budgets estimating to become more accurate.

Future work will look into the Customer relationship in search of ways to speed up decisions, and experiment additional agile practices, such as the use story cards to handle miscommunication issues. The current practices will continue to be refined according to lessons learned on new and existing projects.

5. Acknowledgements

The authors would like to thank the reviewers for their insightful comments. Some of their suggestions were not included exclusively due to time and space constraints, but they will be pursued in subsequent work.

6. References

- [1] Austin, R., 'Project Management: Part I – Methods, Models, and Practices', Benchmark Review, Analyzing IT Metrics for Informed Management Decisions, Vol.4, No.2, Cutter Consortium, 2004.
- [2] Baskerville, R.L. "Investigating Information Systems with Action Research," Communications of the Association for Information Systems (2, Article 19), 1999.
- [3] Baskerville, R.L. and Wood-Harper, A.T. "A Critical Perspective on Action Research as a Method for Information Systems Research," Journal of Information Technology (3:11), 1996, pp. 235-246.
- [4] Berry, D., 'The Inevitable Pain of Software Development, Including of Extreme Programming, Caused by Requirements Volatility', University of Waterloo, Canada, 2002.
- [5] Boehm, B. 'Requirements that Handle IKIWISI, COTS, and Rapid Change', Computer, IEEE, pp.99-102, 2000.

- [6] Boehm, B. and Turner, R., 'Balancing Agility and Discipline – A Guide for the Perplexed' Addison-Wesley, Pearson Education, Boston, USA, 2004.
- [7] Cockburn, A., 'Surviving Object-Oriented Projects', The Agile Software Development Series, Addison-Wesley, 1998.
- [8] Cohn, M. and Ford, D., 'Introducing an Agile Process to an Organization', Cover Feature, Computer Magazine, pp.74-78, 2003.
- [9] Control Chaos, <http://www.controlchaos.com/resources>, current 2005.
- [10] Cordeiro, R., 'Project Life Cycle Management', Critical Software SA, Portugal, 2003.
- [11] Critical Software, S.A., <http://www.criticalsoftware.com>, current 2005.
- [12] Cunha, P. R. and Figueiredo, A. F., 'Action-Research and Critical Rationalism: A Virtuous Marriage', Proceedings of the Xth European Conference on Information Systems, ECIS, Gdansk, Poland, 2002.
- [13] Deusdado, D., "Homens à prova de 'crashes'", Ideias & Negócios, 1998, pp. 30-33.
- [14] Eigner, M., 'Requirements Management and Traceability', An Agile PLM Whitepaper, Agile Software Corporation (current 2005).
- [15] Hull, E., Jackson, K., Dick, J., 'Requirements Engineering – second edition', Springer, 2004.
- [16] Hutinski, Z., Vrcek, N. and Bubas, G., 'Communication Problems in Complex Information System Development Projects', University of Zagreb, Croatia, 2001.
- [17] Larman, C., 'Agile & Iterative Development - A Manager's Guide', Agile Software Development Series, Addison Wesley, 2004.
- [18] Mannion, M. and Keepence, B., 'SMART Requirements', ACM SIGSOFT, SE Notes Vol 20, No. 2, 1995.
- [19] Standish Group, 'Chaos', The Standish Group Inc, West Yarmouth, MA, USA, 1994.
- [20] Standish Group, 'Extreme Chaos', The Standish Group Inc, West Yarmouth, MA, USA, 2001.
- [21] Tomayko, J., 'Engineering of Unstable Requirements Using Agile Methods', Carnegie Mellon University, USA, (current 2005).
- [22] Enterprise Architect, Sparx Systems, Australia, <http://www.sparxsystems.com.au/>